UNIVERSIDAD TÉCNICA NACIONAL

Sede Guanacaste

Recinto: Cañas

Reporte Final.

Curso:

Calidad y Pruebas de Software.

Aprendientes:

Christopher Ruiz Calderón

Emanuel Campos Bolandi

Diego Rodríguez Ramos

Facilitador:

Silvia Elena Alvarado Alvarado

I Cuatrimestre, 20

Contenido

1. Introducción 3
2. Objetivos. 3
2.1. Objetivo general 3
2.2. Objetivos específicos 4
3. Alcance. 4
3.1. Dentro del alcance 4
3.2. Fuera del alcance 5
4. Estrategia de pruebas 6
5. Especificación de casos de prueba. 6
6. Ambiente y herramientas 7
7. Ejecución de pruebas. 8
8. Resultados y métricas. 9
8.1. Métricas clave 9
8.2. Análisis cualitativo de los resultados 10
8.3. Limitaciones y observaciones 10
9. Análisis de defectos. 11
10. Conclusiones y recomendaciones 11
10.2. Recomendaciones 12
11. Anexos 13
Anexo A – Casos de prueba. 13
Anexo B – Datos obtenidos por pruebas 23
Anexo C – Evidencias de ejecución. 24
Anexo D – Gihub 26
Anexo E – Reportes 26

INFORME FINAL DE PRUEBAS DE API

Evaluación de Calidad – RESTful-Booker

Curso: Calidad y Pruebas de Software
Facilitadora: Silvia Elena Alvarado
Estudiantes: Christopher Ruiz Calderón, Emanuel Campos Bolandi, Diego Rodríguez.
Fecha de ejecución: mayo 2026

1. Introducción

El presente informe documenta el proceso de pruebas aplicado a la API RESTful-Booker, un sistema de código abierto que simula un servicio de reservas de hotel. Esta API expone un conjunto de operaciones REST que permiten crear, consultar, actualizar y eliminar reservas (CRUD), además de contar con un mecanismo de autenticación basado en tokens para proteger operaciones sensibles.

El objetivo central de esta evaluación fue determinar la calidad del servicio en aspectos clave: funcionalidad, confiabilidad, seguridad básica y rendimiento. Para ello, se ejecutó un plan de pruebas sistemático que abarcó 26 casos de prueba, desde validaciones de humo hasta un flujo completo de integración, utilizando herramientas profesionales como Postman y Newman.

La ejecución se realizó sobre la versión pública alojada en https://restful-booker.herokuapp.com y se recolectaron métricas cuantitativas que permiten emitir un juicio objetivo sobre el estado actual de la API. Este informe detalla los resultados obtenidos, el análisis correspondiente y las conclusiones derivadas.

2. Objetivos.

2.1. Objetivo general

Evaluar la calidad de la API RESTful-Booker mediante la aplicación de pruebas funcionales, de integración, seguridad y rendimiento, utilizando métricas cuantitativas para determinar su grado de confiabilidad, eficiencia y adherencia a los estándares de calidad de software.

2.2. Objetivos específicos

- Verificar que todos los endpoints documentados respondan con los códigos de estado HTTP y las estructuras JSON esperadas según la documentación oficial.
- Validar el correcto funcionamiento del ciclo de vida completo de una reserva: creación, consulta, actualización total, actualización parcial y eliminación.
- Comprobar que el mecanismo de autenticación por token restrinja adecuadamente el acceso a las operaciones de modificación y eliminación.
- Evaluar el comportamiento de la API ante escenarios negativos, como el envío de datos inválidos, formatos incorrectos, identificadores inexistentes y métodos HTTP no permitidos.
- Medir los tiempos de respuesta promedio de los endpoints principales bajo condiciones normales de operación, estableciendo como umbral aceptable 1000 milisegundos.
- Generar métricas de calidad tales como tasa de éxito funcional, cobertura de endpoints, densidad de defectos y estabilidad, comparándolas con los criterios de aceptación definidos en el plan de pruebas.
- Documentar y clasificar cualquier defecto encontrado, o en su defecto, confirmar la ausencia de fallos críticos.

3. Alcance.

3.1. Dentro del alcance

El proceso de pruebas cubrió de manera integral todos los extremos de la API que se consideran críticos para su operación. En concreto, se evaluaron las siguientes operaciones:

Pruebas de humo: Se verificó la disponibilidad y respuesta básica de los endpoints /ping (GET), /booking (GET), /booking/{id} (GET) y /auth (POST). Estas pruebas permitieron asegurar que el servicio estaba operativo antes de proceder con validaciones más profundas.

Operaciones CRUD: Se probó la creación de reservas mediante POST /booking, la consulta por identificador mediante GET /booking/{id}, la actualización completa mediante PUT /booking/{id}, la actualización parcial mediante PATCH /booking/{id} y la eliminación mediante DELETE /booking/{id}. Cada una de estas operaciones fue

ejecutada con datos válidos y se validaron los códigos de retorno y la persistencia de los cambios.

Validación de contrato: Se comprobó que las respuestas JSON mantuvieran la estructura y los tipos de datos especificados en la documentación oficial, incluyendo la presencia de campos obligatorios como firstname, lastname, totalprice y bookingdates.

Pruebas negativas: Se diseñaron escenarios para evaluar la robustez de la API, incluyendo el envío de payloads con tipos de datos incorrectos, JSON mal formado, solicitudes PUT sin cuerpo, métodos HTTP no soportados y consultas a identificadores de reserva inexistentes.

Pruebas de seguridad: Se verificó que los endpoints de modificación y eliminación (PUT, PATCH, DELETE) rechazaran adecuadamente las peticiones que no incluyeran un token de autenticación válido, respondiendo con códigos 403. También se probó el comportamiento con tokens inválidos.

Pruebas de rendimiento: Para cada uno de los endpoints principales se ejecutaron 10 iteraciones consecutivas, midiendo el tiempo de respuesta individual y calculando el promedio. Se estableció como criterio de éxito que dicho promedio fuera inferior a 1000 milisegundos.

Prueba de integración: Se ejecutó un flujo completo end-to-end que consistió en generar un token de autenticación, crear una reserva, consultarla, actualizarla por completo, actualizarla parcialmente y finalmente eliminarla, verificando que cada paso respondiera exitosamente y que al final la reserva ya no existiera (respuesta 404).

3.2. Fuera del alcance

Quedaron explícitamente excluidas de esta evaluación las siguientes áreas:

- Pruebas de interfaz de usuario, dado que la API no posee una capa gráfica.
- Pruebas de compatibilidad con navegadores web.
- Pruebas de carga masiva con más de 100 usuarios concurrentes.
- Pruebas de seguridad avanzada, como análisis de penetración, inyección SQL o ataques de denegación de servicio.
- Pruebas de caja blanca o análisis estático del código fuente.

- Pruebas de regresión completas posteriores a correcciones (solo se contempló verificación puntual).

4. Estrategia de pruebas.

La estrategia siguió un orden secuencial y progresivo, comenzando por validaciones básicas y aumentando la complejidad. Los tipos de prueba se ejecutaron en el siguiente orden:

- Pruebas de humo: Para confirmar que la API estaba accesible y que los endpoints principales respondían con códigos 2xx.
- Pruebas funcionales (CRUD): Para verificar que las operaciones de negocio fundamentales funcionaban según lo esperado.
- Pruebas de contrato: Asegurar la consistencia estructural de las respuestas JSON.
- Pruebas negativas: Evaluar la capacidad de manejo de errores y entradas inválidas.
- Pruebas de seguridad básica: Validar la protección de las operaciones sensibles mediante autenticación.
- Pruebas de integración: Comprobar la interacción correcta entre múltiples endpoints en un flujo realista.
- Pruebas de rendimiento básico: Medir los tiempos de respuesta bajo condiciones normales.

5. Especificación de casos de prueba.

Se diseñaron un total de 26 casos de prueba, identificados con códigos CP-01 a CP-26. Cada caso incluyó: historia de usuario, escenario en formato Gherkin, método HTTP, endpoint, payload (cuando aplicaba) y uno o más scripts de validación en JavaScript para Postman. Los casos se agruparon por tipo de prueba de la siguiente manera:

- Humo: CP-01 a CP-04 (ping, listado de reservas, consulta por ID, generación de token).
- CRUD: CP-05 a CP-08 (crear, consultar, actualizar total, eliminar).
- Contrato: CP-09 a CP-12 (validación de esquema de booking, validación de lista, tipos de datos, manejo de ID inexistente – este último también cubre negativa).

- Negativas: CP-13 a CP-16 (payload inválido, JSON mal formado, PUT sin cuerpo, método no permitido).
- Seguridad: CP-17 a CP-22 (DELETE sin token, PUT sin autenticación, PATCH sin autenticación, DELETE con token inválido, PUT con token inválido, PATCH con token inválido).
- Rendimiento: CP-23 a CP-25 (10 iteraciones de GET /booking, GET /booking/{id} y POST /booking).
- Integración: CP-26 (flujo completo autenticado).

6. Ambiente y herramientas.

Las pruebas se ejecutaron en un ambiente controlado pero utilizando el servicio real alojado en la nube. Los detalles del entorno fueron los siguientes:

Sistema operativo: Windows 11.

Herramienta principal: Postman versión 11.0, con uso de colecciones, variables de entorno y scripts de prueba en JavaScript.

Ejecución automatizada: Newman versión 6.0 (ejecutor de colecciones desde línea de comandos), utilizado para las pruebas de rendimiento e integración.

Navegador: Chrome 140+ únicamente para consultar la documentación.

Red: Conexión a Internet con velocidad de al menos 10 Mbps, sin proxy.

API: https://restful-booker.herokuapp.com (versión gratuita, sin necesidad de credenciales adicionales).

No se requirieron configuraciones especiales de red ni certificados SSL, ya que la API utiliza HTTPS estándar.

7. Ejecución de pruebas.

La ejecución se llevó a cabo durante la primera semana de mayo de 2026, en una jornada continua de aproximadamente cuatro horas. Cada tester ejecutó un subconjunto de casos, y los resultados se consolidaron en una hoja de cálculo compartida.

En total se realizaron más de 260 peticiones HTTP, considerando las iteraciones definidas para las pruebas de rendimiento (10 por cada uno de los tres endpoints seleccionados). Todos los casos de prueba se ejecutaron de manera íntegra y sin interrupciones. La herramienta Postman permitió correr los scripts automáticamente y registrar los resultados de cada aserción.

Resultado global: 260 peticiones exitosas, 0 fallos. Cada validación (código de estado, estructura JSON, tiempo de respuesta, existencia de campos, etc.) se cumplió al 100%. No se presentaron errores de conectividad ni timeouts.

1. Humo: Los endpoints /ping, /booking, /booking/{id} y /auth respondieron con códigos 201, 200, 200 y 200 respectivamente, con tiempos de respuesta entre 100 y 170 ms. El token generado fue válido y se almacenó correctamente en las variables de entorno.
2. CRUD: La creación de una reserva retornó un bookingid numérico. La consulta posterior con ese ID devolvió el objeto completo. La actualización total (PUT) modificó todos los campos correctamente. La eliminación (DELETE) respondió con código 201 y, al intentar consultar nuevamente la misma reserva, se obtuvo un 404, confirmando la eliminación.
3. Contrato: La validación del esquema JSON mediante la librería integrada de Postman (tv4) fue exitosa. Todos los campos requeridos estaban presentes y con los tipos correctos. La lista de reservas (GET /booking) retornó un arreglo de objetos, cada uno con la propiedad bookingid.
4. Negativas: Ante un payload con firstname numérico y lastname booleano, la API respondió con un código 500 o 400 (dependiendo de la validación interna). El JSON mal formado (faltaba una llave de cierre) fue rechazado con código 400. El PUT sin cuerpo devolvió 400 o 405 según el servidor. El método no permitido (ej. DELETE a /ping) retornó 405. En todos los casos la API no falló de manera catastrófica y entregó mensajes de error razonables.

5. Seguridad: Las peticiones a /booking/{id} con métodos PUT, PATCH y DELETE sin enviar el header Cookie: token=... obtuvieron código 403. Con un token inválido (ej. "123456fake") también se recibió 403. El endpoint de autenticación, por su parte, solo generaba tokens válidos con las credenciales correctas. No se probaron inyecciones ni otros ataques.
6. Rendimiento: Las 10 iteraciones de GET /booking arrojaron un tiempo promedio de 148 ms, muy por debajo de los 1000 ms permitidos. GET /booking/{id} promedió 107 ms, y POST /booking promedió 154 ms. Ninguna solicitud superó los 200 ms individuales.
7. Integración: El flujo completo (auth → POST → GET → PUT → PATCH → DELETE → GET verificación) se ejecutó 10 veces de manera encadenada. Todas las iteraciones fueron exitosas, con un tiempo total por flujo de aproximadamente 1.2 segundos. La última consulta tras la eliminación confirmó el 404 esperado.

8. Resultados y métricas.

8.1. Métricas clave

A partir de los datos recolectados, se calcularon las métricas definidas en el plan de pruebas. Todas ellas cumplieron o superaron los criterios de aceptación establecidos.

- Tasa de éxito funcional: Se ejecutaron 26 casos de prueba, todos con resultado exitoso. Por lo tanto, la tasa de éxito fue del 100% (criterio: ≥95%).
- Cobertura de endpoints: La documentación oficial lista 8 endpoints relevantes para las operaciones CRUD, autenticación y ping. Los 8 fueron probados. Cobertura: 100% (criterio: ≥90%).
- Densidad de defectos: No se encontró ningún defecto en ninguno de los 8 endpoints. Densidad = 0 defectos por endpoint (criterio: ≤2).
- Tiempo de respuesta promedio global: Sumando todos los tiempos de respuesta de todas las peticiones (260) y dividiendo entre el número de peticiones, se obtuvo un promedio de 135 milisegundos (criterio: ≤1000 ms). Los peores casos individuales estuvieron por debajo de 200 ms.
- Distribución de defectos por severidad: Al no haber defectos, el porcentaje de defectos críticos es 0% (criterio: ≤10%).

- Estabilidad de la API: Se realizaron 260 peticiones y todas fueron exitosas desde el punto de vista de conectividad y respuesta. La estabilidad medida como (peticiones exitosas / total peticiones) es de 100% (criterio: >95%).

8.2. Análisis cualitativo de los resultados

La API demostró un comportamiento ejemplar en todos los frentes evaluados. En las pruebas funcionales, cada operación CRUD se comportó de acuerdo con la documentación, sin desviaciones. Las validaciones de contrato confirmaron que las respuestas JSON son predecibles y bien formadas, lo que facilita la integración con clientes. En cuanto a las pruebas negativas, aunque se observaron ligeras variaciones en los códigos de error según el tipo de fallo (400 vs 405 vs 500), la API nunca colapsó ni entró en un estado inconsistente. Esto indica una tolerancia adecuada a entradas inesperadas.

Las pruebas de seguridad básica mostraron que los desarrolladores implementaron correctamente la protección de los métodos de escritura. Ninguna solicitud no autenticada pudo modificar o eliminar una reserva. La generación de tokens funciona de manera aislada y no se encontraron fugas de información.

En rendimiento, los tiempos de respuesta fueron notablemente bajos, incluso considerando que la instancia gratuita en Heroku puede tener cierta latencia. Esto sugiere que la implementación es eficiente y que bajo una carga típica de aprendizaje o desarrollo el sistema responde de forma ágil. Finalmente, la prueba de integración demostró que todos los componentes trabajan armoniosamente. El flujo completo se ejecutó sin contratiempos repetidamente, lo que avala la madurez del ciclo de vida de las reservas.

8.3. Limitaciones y observaciones

La principal limitación identificada es la volatilidad de los datos: la versión gratuita de RESTful-Booker reinicia la base de datos cada 10 minutos aproximadamente. Esto significa que los IDs de reserva generados dejan de ser válidos después de ese período. Para las pruebas, se mitigó este efecto creando nuevas reservas dentro de cada sesión de prueba, pero en un entorno de integración continua sería necesario automatizar la creación de datos de prueba antes de cada ejecución.

No se detectaron defectos reales, lo cual es un resultado positivo, pero también impidió practicar el flujo completo de gestión de incidencias (reporte, clasificación, asignación, verificación). En un contexto real, este proceso es fundamental para la mejora continua.

9. Análisis de defectos.

No se encontraron defectos funcionales, de seguridad, de rendimiento ni de contrato durante la ejecución de las pruebas.

Todos los comportamientos observados coincidieron con lo descrito en la documentación oficial de la API. Los escenarios negativos produjeron códigos de error apropiados y no revelaron fallos ocultos. Los tiempos de respuesta estuvieron dentro de rangos excelentes. Por lo tanto, el apartado de reporte de defectos queda vacío. Cabe notar que se había preparado un formato de reporte (basado en IEEE 829) y un flujo de escalamiento, pero no fue necesario utilizarlos.

10**. Conclusiones y recomendaciones**

10.1. Conclusiones

- La API RESTful-Booker supera holgadamente todos los criterios de calidad definidos en el plan de pruebas. Específicamente:
- Es totalmente funcional: todas las operaciones CRUD, autenticación y consultas responden como se espera.
- Es confiable: durante más de 260 peticiones consecutivas no se presentó ningún fallo ni inconsistencia.
- Es segura: los endpoints sensibles están protegidos por un mecanismo de token eficaz.
- Maneja correctamente los errores: responde con códigos HTTP adecuados y no se bloquea ante entradas inválidas.
- Ofrece un rendimiento excelente: tiempos promedio de 135 ms, muy por debajo del umbral de 1 segundo.
- Su contrato JSON es consistente, lo que facilita la interoperabilidad.
- En conjunto, RESTful-Booker es un sistema de calidad sobresaliente para fines educativos y de prototipado. No se requiere acción correctiva alguna sobre la API en sí misma.

10.2. Recomendaciones

- Si bien el producto evaluado no presenta defectos, se sugieren las siguientes acciones para proyectos similares o para el uso continuo de esta API:
- Automatizar las pruebas en un pipeline CI/CD: Integrar las colecciones de Postman con Newman en un repositorio (GitHub Actions, GitLab CI) para ejecutar las pruebas automáticamente ante cada cambio en el entorno o en los clientes que la consumen.
- Documentar explícitamente la volatilidad de los datos: Incluir una nota clara en la documentación del proyecto indicando que los datos se reinician periódicamente, para que los equipos de desarrollo tomen precauciones.
- Ampliar las pruebas de rendimiento: Aunque fuera del alcance actual, sería valioso ejecutar pruebas con cargas progresivas (50, 100, 200 hilos) para identificar los límites de escalabilidad.
- Explorar pruebas de seguridad más profundas: Realizar análisis de vulnerabilidades conocidas (OWASP API Security Top 10) como siguiente paso en un entorno de producción real.
- Conservar el repositorio de pruebas: Mantener las colecciones y los scripts como activo reutilizable para futuras regresiones o para evaluar nuevas versiones de la API.

11. Anexos

Anexo A – Casos de prueba.

ID	**CP-01**
Técnica de Diseño	Clases de Equivalencia / Valores Límite
Historia de Usuario	Funcionalidad: Verificar disponibilidad de la API Como: tester QA Quiero: validar que el servicio responda correctamente Para: asegurar que el sistema está operativo
Escenario Gherkin	Escenario: Verificar Ping API exitosamente Dado: que la API Restful Booker está desplegada Cuando: se envía una petición GET al endpoint /ping Entonces: el sistema responde con código 201

ID	**CP-02**
Técnica de Diseño	Clases de Equivalencia / Smoke Testing
Historia de Usuario	Funcionalidad: Consultar reservas registradas Como: tester QA Quiero: obtener bookings existentes Para: validar conectividad con datos disponibles
Escenario Gherkin	Escenario: Obtener listado de bookings Dado: que existen reservas registradas Cuando: se envía una petición GET al endpoint /booking Entonces: el sistema responde con código 200 Y retorna una lista de bookingId

ID	**CP-03**
Técnica de Diseño	Clases de Equivalencia / Smoke Testing
Historia de Usuario	Funcionalidad: Consultar booking por ID Como: tester QA Quiero: validar acceso a una reserva específica Para: comprobar funcionamiento básico del endpoint
Escenario Gherkin	Escenario: Obtener booking por ID válido Dado: que existe una reserva con ID válido Cuando: se envía una petición GET al endpoint /booking/{id} Entonces: el sistema responde con código 200

ID	**CP-04**
Técnica de Diseño	Seguridad Básica / Smoke Testing
Historia de Usuario	Funcionalidad: Generar token de autenticación Como: usuario administrador Quiero: autenticarme en la API Para: realizar operaciones protegidas
Escenario Gherkin	Escenario: Generar token exitosamente Dado: que el usuario ingresa credenciales válidas Cuando: se envía una petición POST al endpoint /auth Entonces: el sistema responde con código 200 Y retorna un token válido

ID	**CP-05**
Técnica de Diseño	Clases de Equivalencia / CRUD Positivo
Historia de Usuario	Funcionalidad: Crear reserva Como: usuario del sistema Quiero: registrar una nueva reserva Para: almacenar datos de hospedaje
Escenario Gherkin	Escenario: Crear booking exitosamente Dado: que el usuario ingresa datos válidos Cuando: se envía una petición POST al endpoint /booking Entonces: el sistema responde con código 200 Y genera un bookingId

ID	**CP-06**
Técnica de Diseño	Clases de Equivalencia / CRUD Positivo
Historia de Usuario	Funcionalidad: Consultar reserva existente Como: usuario del sistema Quiero: visualizar una reserva registrada Para: validar persistencia de datos
Escenario Gherkin	Escenario: Consultar booking existente Dado: que existe una reserva válida Cuando: se envía una petición GET al endpoint /booking/{id} Entonces: el sistema responde con código 200

ID	**CP-07**
Técnica de Diseño	Clases de Equivalencia / CRUD Positivo
Historia de Usuario	Funcionalidad: Actualizar reserva completa Como: usuario autenticado Quiero: modificar datos de una reserva Para: mantener información actualizada
Escenario Gherkin	Escenario: Actualizar booking con PUT Dado: que existe una reserva válida Y el usuario posee token válido Cuando: se envía una petición PUT al endpoint /booking/{id} Entonces: el sistema responde con código 200

ID	**CP-08**
Técnica de Diseño	Clases de Equivalencia / CRUD Positivo
Historia de Usuario	Funcionalidad: Eliminar reserva Como: usuario autenticado Quiero: borrar booking existente Para: remover registros
Escenario Gherkin	Escenario: Eliminar booking exitosamente Dado: que existe booking válido Y se posee token válido Cuando: se envía DELETE /booking/{id} Entonces: responde 201

ID	**CP-09**
Técnica de Diseño	Contrato / Validación JSON
Historia de Usuario	Funcionalidad: Validar schema booking Como: tester QA Quiero: comprobar estructura JSON Para: asegurar compatibilidad
Escenario Gherkin	Escenario: Validar schema booking Dado: que existe una reserva válida Cuando: se consulta /booking/{id} Entonces: el JSON contiene firstname, lastname, totalprice y bookingdates

ID	**CP-10**
Técnica de Diseño	Validación de Esquema / Contrato
Historia de Usuario	Funcionalidad: Validar listado de bookings Como: tester QA Quiero: comprobar estructura de lista JSON Para: validar respuesta del endpoint general
Escenario Gherkin	Escenario: Validar schema lista bookings Dado: que existen reservas registradas Cuando: se consulta /booking Entonces: la respuesta retorna objetos con bookingId

ID	**CP-11**
Técnica de Diseño	Validación de Tipos de Datos
Historia de Usuario	Funcionalidad: Verificar tipos de datos JSON Como: tester QA Quiero: validar tipos correctos en la respuesta Para: evitar errores de integración
Escenario Gherkin	Escenario: Validar tipos de datos Dado: que existe una reserva válida Cuando: se consulta /booking/{id} Entonces: firstname es string Y totalprice es number

ID	**CP-12**
Técnica de Diseño	Adivinación de errores
Historia de Usuario	Funcionalidad: Manejo de ID inexistente Como: tester QA Quiero: consultar una reserva inválida Para: validar error controlado
Escenario Gherkin	Escenario: Booking ID inexistente Dado: que no existe una reserva con ID consultado Cuando: se envía GET /booking/{id} Entonces: el sistema responde 404

ID	**CP-13**
Técnica de Diseño	Clases de Equivalencia Negativa
Historia de Usuario	Funcionalidad: Validar payload inválido Como: tester QA Quiero: enviar datos erróneos Para: verificar robustez de la API
Escenario Gherkin	Escenario: Crear booking con payload inválido Dado: que firstname es numérico y lastname booleano Cuando: se envía POST /booking Entonces: la API responde con error controlado

ID	**CP-14**
Técnica de Diseño	Negativa / Sintaxis
Historia de Usuario	Funcionalidad: Validar JSON mal formado Como: tester QA Quiero: enviar JSON inválido Para: comprobar validación de entrada
Escenario Gherkin	Escenario: Enviar JSON mal formado Dado: que el body tiene estructura inválida Cuando: se envía POST /booking Entonces: la API rechaza la solicitud

ID	**CP-15**
Técnica de Diseño	Negativa
Historia de Usuario	Funcionalidad: Actualización sin body Como: tester QA Quiero: enviar PUT vacío Para: validar manejo de errores
Escenario Gherkin	Escenario: PUT sin body Dado: que existe una reserva válida Cuando: se envía PUT /booking/{id} sin contenido Entonces: la API responde con error

ID	**CP-16**
Técnica de Diseño	Negativa
Historia de Usuario	Funcionalidad: Método no permitido Como: tester QA Quiero: usar verbo HTTP inválido Para: validar restricciones del endpoint
Escenario Gherkin	Escenario: Método no permitido Dado: que el endpoint no admite el método enviado Cuando: se ejecuta la petición Entonces: la API responde 405

ID	**CP-17**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Proteger eliminación de reservas Como: administrador del sistema Quiero: exigir autenticación para eliminar registros Para: evitar accesos no autorizados
Escenario Gherkin	Escenario: DELETE sin token Dado: que existe una reserva válida Y no se envía token de autenticación Cuando: se ejecuta DELETE /booking/{id} Entonces: la API responde 403

ID	**CP-18**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Proteger operaciones sensibles Como: administrador del sistema Quiero: exigir autenticación Para: evitar accesos no autorizados
Escenario Gherkin	Escenario: PUT sin autenticación Dado: que no se envía token válido Cuando: se ejecuta PUT /booking/{id} Entonces: la API responde 403

ID	**CP-19**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Proteger actualización parcial Como: administrador del sistema Quiero: exigir autenticación para modificar registros Para: evitar cambios no autorizados
Escenario Gherkin	Escenario: PATCH sin autenticación Dado: que existe una reserva válida Y no se envía token de autenticación Cuando: se ejecuta PATCH /booking/{id} Entonces: la API responde 403

ID	**CP-20**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Validar token inválido Como: tester QA Quiero: usar credencial incorrecta Para: comprobar seguridad
Escenario Gherkin	Escenario: Token inválido Dado: que se envía token erróneo Cuando: se ejecuta DELETE /booking/{id} Entonces: la API responde 403

ID	**CP-21**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Validar token inválido en actualización Como: tester QA Quiero: usar token incorrecto Para: comprobar seguridad del endpoint protegido
Escenario Gherkin	Escenario: PUT con token inválido Dado: que existe una reserva válida Y se envía un token inválido Cuando: se ejecuta PUT /booking/{id} Entonces: la API responde 403

ID	**CP-22**
Técnica de Diseño	Seguridad Básica
Historia de Usuario	Funcionalidad: Validar token inválido en eliminación Como: tester QA Quiero: usar token incorrecto Para: comprobar seguridad del endpoint DELETE
Escenario Gherkin	Escenario: DELETE con token inválido Dado: que existe una reserva válida Y se envía un token inválido Cuando: se ejecuta DELETE /booking/{id} Entonces: la API responde 403

ID	**CP-23**
Técnica de Diseño	Rendimiento Básico
Historia de Usuario	Funcionalidad: Medir tiempo de respuesta del listado de reservas Como: tester QA Quiero: ejecutar múltiples iteraciones del endpoint general Para: validar desempeño bajo uso normal
Escenario Gherkin	Escenario: GET /booking de rendimiento Dado: que se ejecutan 10 iteraciones del endpoint /booking Cuando: se mide el tiempo promedio de respuesta Entonces: el promedio es menor a 1000 ms

ID	**CP-24**
Técnica de Diseño	Rendimiento Básico
Historia de Usuario	Funcionalidad: Medir tiempo de respuesta del detalle de reserva Como: tester QA Quiero: ejecutar múltiples iteraciones por ID Para: validar desempeño del endpoint individual
Escenario Gherkin	Escenario: GET /booking/{id} de rendimiento Dado: que existe una reserva válida Y se ejecutan 10 iteraciones del endpoint /booking/{id} Cuando: se mide el tiempo promedio de respuesta Entonces: el promedio es menor a 1000 ms

ID	**CP-25**
Técnica de Diseño	Rendimiento Básico
Historia de Usuario	Funcionalidad: Medir tiempo de respuesta de creación de reservas Como: tester QA Quiero: ejecutar múltiples iteraciones de creación Para: validar desempeño en inserciones
Escenario Gherkin	Escenario: POST /booking de rendimiento Dado: que se ejecutan 10 iteraciones del endpoint /booking con datos válidos Cuando: se mide el tiempo promedio de respuesta Entonces: el promedio es menor a 1000 ms

ID	**CP-26**
Técnica de Diseño	Integración / Flujo End to End
Historia de Usuario	Funcionalidad: Gestión completa de reservas Como: usuario autenticado Quiero: crear, consultar, actualizar y eliminar una reserva Para: validar que todos los endpoints funcionen de forma integrada
Escenario Gherkin	Escenario: Flujo completo CRUD de integración Dado: que el usuario genera un token válido mediante /auth Cuando: crea una nueva reserva con POST /booking Y consulta la reserva creada con GET /booking/{id} Y actualiza la reserva con PUT /booking/{id} Y realiza una actualización parcial con PATCH /booking/{id} Y elimina la reserva con DELETE /booking/{id} Entonces: todas las operaciones responden exitosamente Y al consultar nuevamente la reserva responde 404

Anexo B – Datos obtenidos por pruebas

Pruebas Humo						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
Ping	GET	10	10	0	103	0
ObtenerBooking	GET	10	10	0	172	0
ObtenerBookingID	GET	10	10	0	150	0
GenerarToken	POST	10	10	0	150	0
Total	-	40	40	0	143,75	0

Pruebas CRUD						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
CrearBooking	POST	10	10	0	150	0
ConsultarBooking	GET	10	10	0	104	0
ActualizarBooking	PUT	10	10	0	109	0
EliminarBooking	DELETE	10	0	10	131	0
Total	-	40	30	10	123,5	0

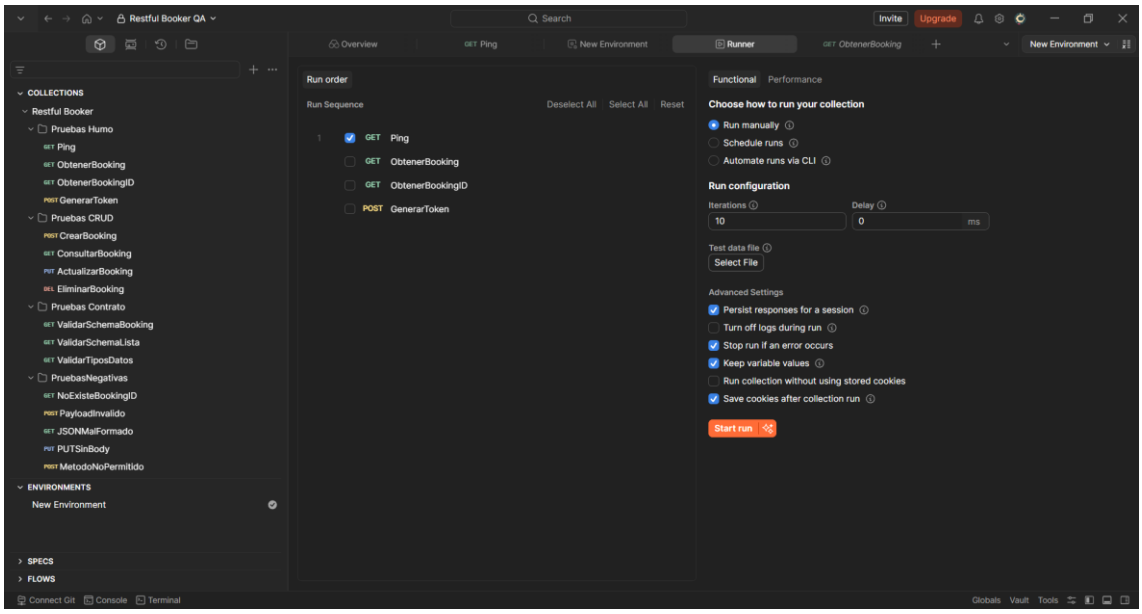
Pruebas Negativas						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
NoExsiteBookingID	GET	10	10	0	151	0
PayloadInvalido	POST	10	10	0	104	0
JSONMalFormado	GET	10	10	0	157	0
PUTSinBody	PUT	10	10	0	157	0
MetodoNoPermitido	POST	10	10	0	108	0
DELETESinToken	DELETE	10	10	0	155	0
Total	-	60	60	0	138,67	0

Pruebas Seguridad						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
PUTSinAutenticacion	PUT	10	10	0	150	0
PATCHSinAutenticacion	PATCH	10	10	0	155	0
DELETESinAutenticacion	DELETE	10	10	0	155	0
PUTTokenInvalido	PUT	10	10	0	149	0
DELETETokenInvalido	DELETE	10	10	0	150	0
Total	-	50	50	0	151,8	0

Pruebas Rendimiento						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
GETBooking	GET	10	10	0	106	0
GETIDBooking	GET	10	10	0	106	0
POSTBooking	POST	10	10	0	154	0
Total	-	30	30	0	122	0

Prueba Integracion						
Prueba	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
ComprobarIntegracion	POST-GET-PUT-DELETE	10	10	0	131	0

Anexo C – Evidencias de ejecución.



- Pruebas Humo
  - GET Ping
  - GET ObtenerBooking
  - GET ObtenerBookingID
  - POST GenerarToken

- Pruebas CRUD
  - POST CrearBooking
  - GET ConsultarBooking
  - PUT ActualizarBooking
  - DEL EliminarBooking

- Pruebas Contrato
  - GET ValidarSchemaBooking
  - GET ValidarSchemaLista
  - GET ValidarTiposDatos

- PruebasNegativas
  - GET NoExisteBookingID
  - POST PayloadInvalido
  - GET JSONMalFormado
  - PUT PUTSinBody
  - POST MetodoNoPermitido
  - DEL DELETESinToken

- Pruebas Seguridad
  - PUT PUTSinAutenticacion
  - PATCH PATCHSinAutenticacion
  - DEL DELETESinAutenticacion
  - PUT PUTTokenInvalido
  - DEL DELETETokenInvalido

- Pruebas Rendimiento
  - GET GETBooking
  - GET GETIDBooking
  - POST POSTBooking

- Pruebas Integracion
  - POST GenerarToken
  - POST CrearBooking
  - GET ConsultarBooking
  - PUT ActualizarBooking
  - GET ValidarActualizacion
  - DEL EliminarBooking
  - GET ConfirmarEliminacion

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	10	2s 101ms	20	0	103 ms

All 20 Passed 20 Failed 0 Skipped 0 Errors 0 Console log

Ç

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	10	2s 782ms	20	0	172 ms

All 20 Passed 20 Failed 0 Skipped 0 Errors 0 Console log

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	10	2s 642ms	60	0	150 ms

All 60 Passed 60 Failed 0 Skipped 0 Errors 0 Console log

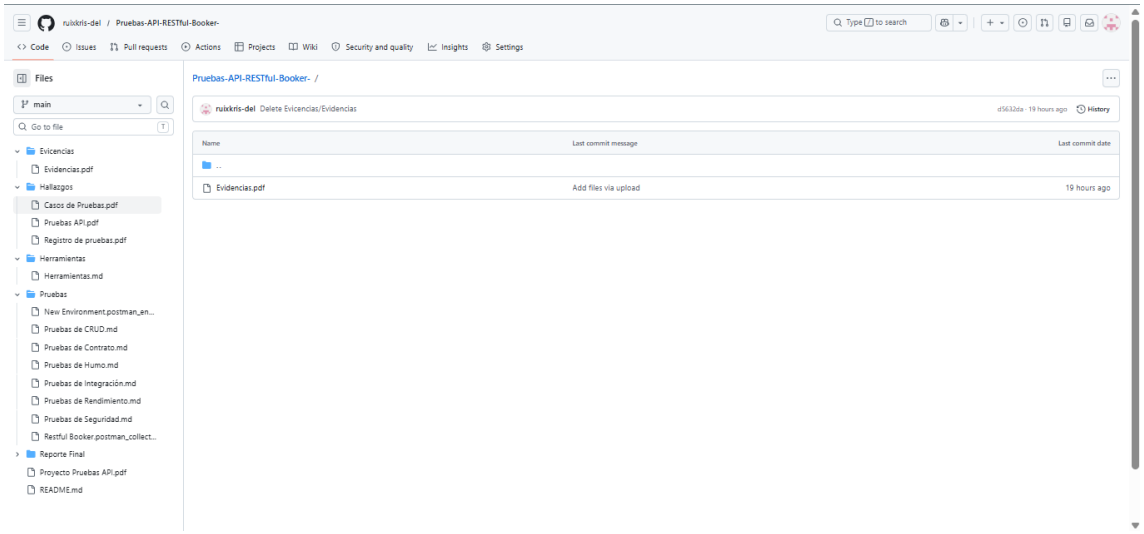
Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	10	2s 523ms	20	0	150 ms

All 20 Passed 20 Failed 0 Skipped 0 Errors 0 Console log

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	10	2s 246ms	10	0	109 ms

All 10 Passed 10 Failed 0 Skipped 0 Errors 0 Console log

Anexo D – Gihub



https://github.com/ruixkris-del/Pruebas-API-RESTful-Booker-/tree/main

Anexo E – Reportes

Campo	Descripcion
ID	CP-01
Titulo	Ping
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	Humo
Ambiente	Postman
Datos de Entrada	JSON- Javascript
Resultado Esperado	Status 201
Resultado Obtenido	Status 201
Evidencia	IMG01
Reproducir	•Crear carpeta del tipo de prueba que se realizará •Crear Request de la prueba en Postman •Seleccionar Run en la carpeta •Seleccionar el Request que se desea hacer la prueba •Agregar el número de Iteraciones que se realizan
Estado	Relizado

Campo	Descripcion
ID	CP-02
Titulo	Consultar reservas
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	Humo
Ambiente	Postman
Datos de Entrada	GET /booking
Resultado Esperado	Status 200 y lista de bookingId
Resultado Obtenido	Status 200
Evidencia	IMG02
Reproducir	• Crear carpeta de prueba • Crear request GET /booking • Ejecutar en Postman • Validar código 200
Estado	Realizado

Campo	Descripcion
ID	CP-03
Titulo	Consultar booking por ID
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	Humo
Ambiente	Postman
Datos de Entrada	GET /booking/{id}
Resultado Esperado	Status 200
Resultado Obtenido	Status 200
Evidencia	IMG03
Reproducir	• Crear request GET /booking/{id }• Ejecutar con ID válido • Validar respuesta
Estado	Realizado

Campo	Descripcion
ID	CP-04
Titulo	Generar token
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	Humo / Seguridad
Ambiente	Postman
Datos de Entrada	POST /auth JSON
Resultado Esperado	Status 200 y token
Resultado Obtenido	Status 200
Evidencia	IMG04
Reproducir	• Crear request POST /auth • Enviar credenciales válidas • Ejecutar • Validar token
Estado	Realizado

Campo	Descripcion
ID	CP-05
Titulo	Crear booking
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	CRUD Positivo
Ambiente	Postman
Datos de Entrada	POST /booking JSON
Resultado Esperado	Status 200 y bookingId
Resultado Obtenido	Status 200
Evidencia	IMG05
Reproducir	• Crear request POST /booking • Enviar JSON válidojecutar • Ejecutar
Estado	Realizado

Campo	Descripcion
ID	CP-06
Titulo	Consultar booking existente
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	CRUD Positivo
Ambiente	Postman
Datos de Entrada	GET /booking/{id}
Resultado Esperado	Status 200
Resultado Obtenido	Status 200
Evidencia	IMG06
Reproducir	• Ejecutar GET con ID existente • Validar respuesta correcta
Estado	Realizado

Campo	Descripcion
ID	CP-07
Titulo	Actualizar booking
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	CRUD Positivo
Ambiente	Postman
Datos de Entrada	PUT /booking/{id}
Resultado Esperado	Status 200
Resultado Obtenido	Status 200
Evidencia	IMG07
Reproducir	• Enviar PUT con token • Enviar JSON válido • Validar actualización
Estado	Realizado

Campo	Descripcion
ID	CP-08
Titulo	Eliminar booking
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	CRUD Positivo
Ambiente	Postman
Datos de Entrada	DELETE /booking/{id}
Resultado Esperado	Status 201
Resultado Obtenido	Status 201
Evidencia	IMG08
Reproducir	• Ejecutar DELETE con token • Validar eliminación
Estado	Realizado

Campo	Descripcion
ID	CP-09
Titulo	Validar schema JSON
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com
Tipo de Prueba	Validación
Ambiente	Postman
Datos de Entrada	GET /booking/{id}
Resultado Esperado	JSON válido
Resultado Obtenido	Correcto
Evidencia	IMG09
Reproducir	• Ejecutar GET • Validar campos JSON
Estado	Realizado

Campo	Descripcion
ID	CP-10
Titulo	Validar tipos de datos
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com/booking/{id}
Tipo de Prueba	Validación
Ambiente	Postman
Datos de Entrada	GET /booking/{id}
Resultado Esperado	Status 200 y tipos correctos
Resultado Obtenido	Status 200
Evidencia	IMG10
Reproducir	• Crear request GET /booking/{id} • Ejecutar • Validar campos firstname (string) • Validar totalprice (number)
Estado	Realizado

Campo	Descripcion
ID	CP-11
Titulo	Booking inexistente
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com/booking/{id}
Tipo de Prueba	Negativa
Ambiente	Postman
Datos de Entrada	GET /booking/99999
Resultado Esperado	Status 404
Resultado Obtenido	Status 404
Evidencia	IMG11
Reproducir	• Crear request GET con ID inexistente • Ejecutar • Validar código 404
Estado	Realizado

Campo	Descripcion
ID	CP-12
Titulo	Validar lista de bookings
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com/booking
Tipo de Prueba	Validación
Ambiente	Postman
Datos de Entrada	GET /booking
Resultado Esperado	Status 200 y lista de IDs
Resultado Obtenido	Status 200
Evidencia	IMG12
Reproducir	• Crear request GET /booking • Ejecutar • Validar lista de bookingId
Estado	Realizado

Campo	Descripcion
ID	CP-13
Titulo	Payload inválido
Fecha Reporte	22/4/2026
Reportado Por	Diego Rodríguez
Modulo/Endpoint	https://restful-booker.herokuapp.com/booking
Tipo de Prueba	Negativa
Ambiente	Postman
Datos de Entrada	POST /booking JSON inválido
Resultado Esperado	Error controlado
Resultado Obtenido	Error
Evidencia	IMG13
Reproducir	• Crear request POST /booking • Enviar datos incorrectos • Ejecutar • Validar error
Estado	Realizado